

Lab 6:

Manasa ST

MCA-A

2547133

Aim

To implement Logistic Regression and K Nearest Neighbors (KNN) classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset and compare their performance using standard classification evaluation metrics.

1. Import Libraries

```
In [22]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report
)

sns.set_style("whitegrid")
```

2. Load the Dataset

Dataset downloaded from the UCI repository and stored locally as `data/wdbc.data` (no header) and column names attached manually as per `data/wdbc.names`.

```
In [23]: DATA_PATH = "wdbc.data"

columns = ["id", "diagnosis"]
stats = ["mean", "se", "worst"]
features = ["radius", "texture", "perimeter", "area", "smoothness",
            "compactness", "concavity", "concave_points", "symmetry", "fr

for stat in stats:
    for feat in features:
        columns.append(f"{feat}_{stat}")

df = pd.read_csv(DATA_PATH, header=None, names=columns)
```

```
print(f"Shape: {df.shape}")
df.head()
```

Shape: (569, 32)

Out[23]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smooth
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns

3. Exploratory Data Analysis

In [24]: `df.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     569 non-null    int64
1   diagnosis                             569 non-null    str
2   radius_mean                           569 non-null    float64
3   texture_mean                           569 non-null    float64
4   perimeter_mean                         569 non-null    float64
5   area_mean                             569 non-null    float64
6   smoothness_mean                       569 non-null    float64
7   compactness_mean                      569 non-null    float64
8   concavity_mean                        569 non-null    float64
9   concave_points_mean                  569 non-null    float64
10  symmetry_mean                         569 non-null    float64
11  fractal_dimension_mean                569 non-null    float64
12  radius_se                             569 non-null    float64
13  texture_se                             569 non-null    float64
14  perimeter_se                           569 non-null    float64
15  area_se                               569 non-null    float64
16  smoothness_se                         569 non-null    float64
17  compactness_se                        569 non-null    float64
18  concavity_se                          569 non-null    float64
19  concave_points_se                     569 non-null    float64
20  symmetry_se                           569 non-null    float64
21  fractal_dimension_se                  569 non-null    float64
22  radius_worst                          569 non-null    float64
23  texture_worst                         569 non-null    float64
24  perimeter_worst                       569 non-null    float64
25  area_worst                            569 non-null    float64
26  smoothness_worst                     569 non-null    float64
27  compactness_worst                     569 non-null    float64
28  concavity_worst                       569 non-null    float64
29  concave_points_worst                  569 non-null    float64
30  symmetry_worst                        569 non-null    float64
31  fractal_dimension_worst                569 non-null    float64
dtypes: float64(30), int64(1), str(1)
memory usage: 142.4 KB

```

In [25]: `df.describe()`

```

Out[25]:

```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothn
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	5
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	

8 rows x 31 columns

3.1 Check for Missing Values

```
In [26]: print(df.isnull().sum().sum(), "missing values in the dataset")
print(df.duplicated().sum(), "duplicate rows in the dataset")
```

0 missing values in the dataset
0 duplicate rows in the dataset

3.2 Class Distribution

```
In [27]: print(df["diagnosis"].value_counts())

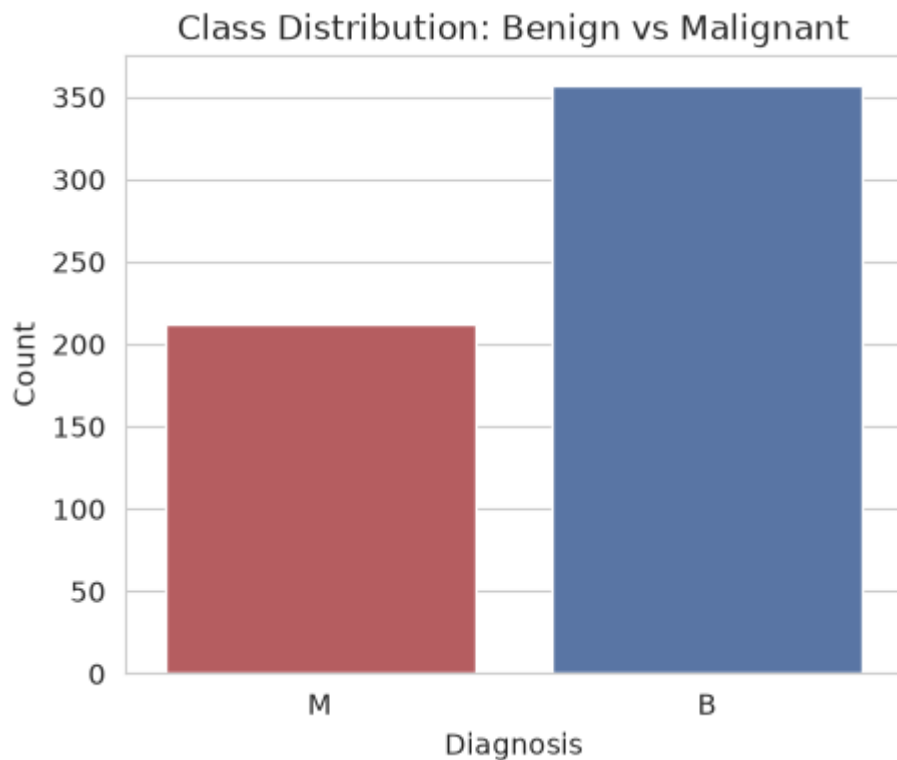
plt.figure(figsize=(5, 4))
sns.countplot(x="diagnosis", data=df, palette={"B": "#4C72B0", "M": "#C44E52"})
plt.title("Class Distribution: Benign vs Malignant")
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.show()
```

```
diagnosis
B      357
M      212
Name: count, dtype: int64
```

/tmp/ipykernel_36049/3242398149.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

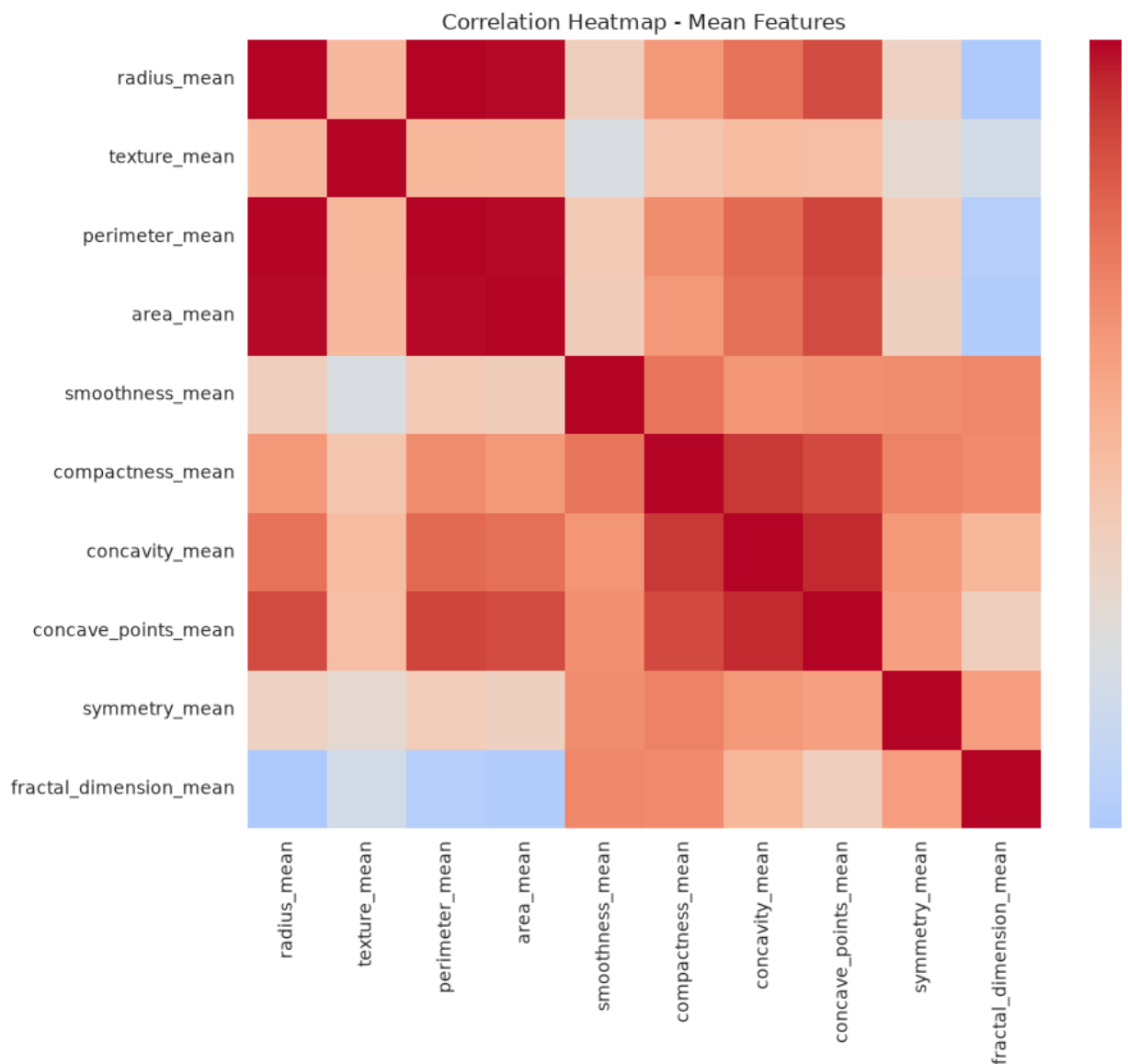
```
sns.countplot(x="diagnosis", data=df, palette={"B": "#4C72B0", "M": "#C44E52"}, hue="diagnosis", legend=False)
```



3.3 Correlation Heatmap (mean features)

```
In [28]: mean_cols = [c for c in df.columns if c.endswith("_mean")]

plt.figure(figsize=(10, 8))
sns.heatmap(df[mean_cols].corr(), cmap="coolwarm", center=0, annot=False)
plt.title("Correlation Heatmap - Mean Features")
plt.show()
```



4. Data Preprocessing

- Drop `id` column (not a predictive feature).
- Encode `diagnosis` : M -> 1, B -> 0.
- Separate features (X) and target (y).

```
In [29]: df_proc = df.drop(columns=["id"]).copy()

le = LabelEncoder()
df_proc["diagnosis"] = le.fit_transform(df_proc["diagnosis"]) # B=0, M=1
print("Classes:", dict(zip(le.classes_, le.transform(le.classes_))))

X = df_proc.drop(columns=["diagnosis"])
y = df_proc["diagnosis"]
```

```
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
```

```
Classes: {'B': np.int64(0), 'M': np.int64(1)}
X shape: (569, 30)
y shape: (569,)
```

5. Train-Test Split

```
In [30]: X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
        )

print(f"Train set: {X_train.shape}, Test set: {X_test.shape}")
```

```
Train set: (455, 30), Test set: (114, 30)
```

6. Feature Scaling

KNN is distance-based, so features must be scaled. Logistic Regression also converges better on data. Scaler is fit only on train data to avoid leakage.

```
In [31]: scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaling done. Mean ~0, Std ~1 on train set:")
print(f"Train mean: {X_train_scaled.mean():.4f}, Train std: {X_train_scaled.std():.4f}")
```

```
Scaling done. Mean ~0, Std ~1 on train set:
Train mean: 0.0000, Train std: 1.0000
```

7. Train Logistic Regression Classifier

```
In [32]: log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

y_pred_log = log_reg.predict(X_test_scaled)
print("Logistic Regression trained.")
```

```
Logistic Regression trained.
```

8. Train K Nearest Neighbors (KNN) Classifier

8.1 Choosing k

Testing $k = 1$ to 20 on the test set to pick the best performing k .

```
In [33]: k_range = range(1, 21)
k_accuracies = []

for k in k_range:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
```

```

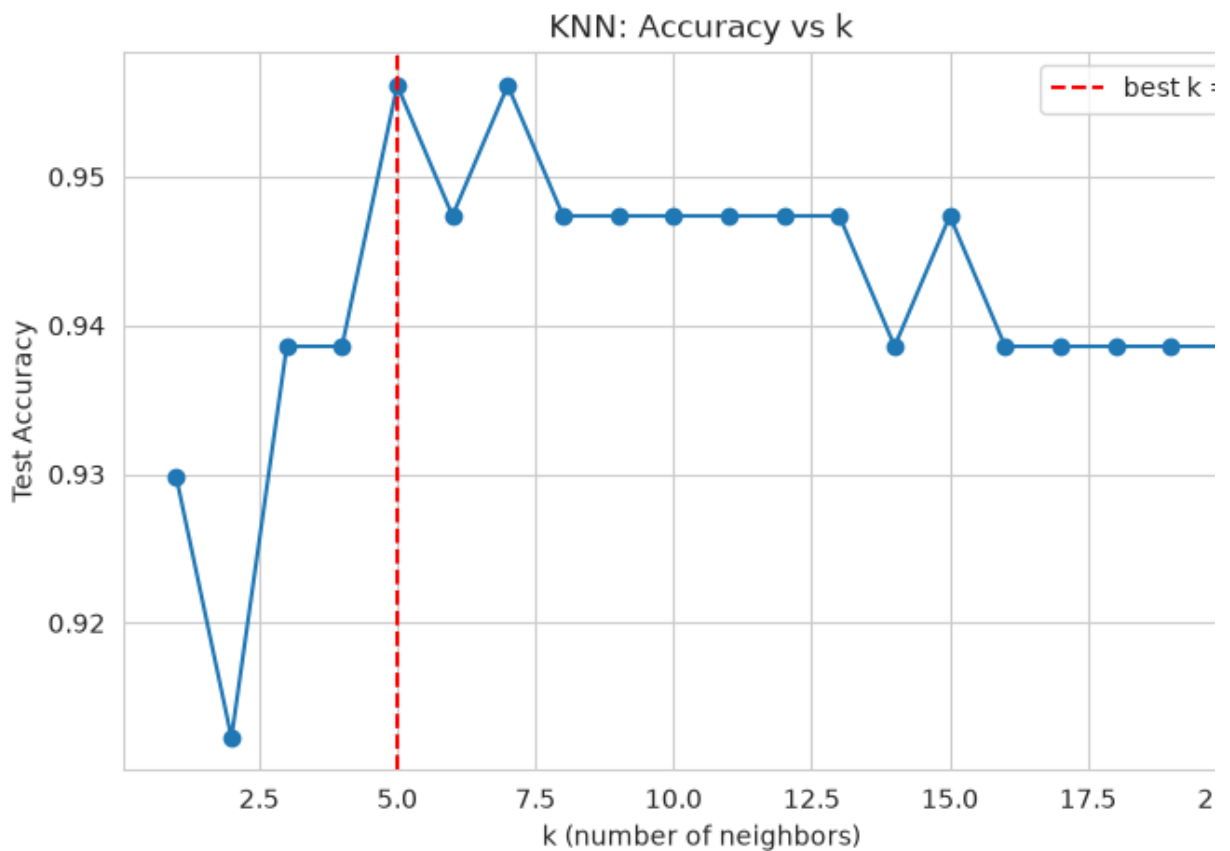
knn_temp.fit(X_train_scaled, y_train)
acc = accuracy_score(y_test, knn_temp.predict(X_test_scaled))
k_accuracies.append(acc)

best_k = list(k_range)[np.argmax(k_accuracies)]
print(f"Best k: {best_k}, Accuracy: {max(k_accuracies):.4f}")

plt.figure(figsize=(8, 5))
plt.plot(list(k_range), k_accuracies, marker="o")
plt.axvline(best_k, color="red", linestyle="--", label=f"best k = {best_k}")
plt.xlabel("k (number of neighbors)")
plt.ylabel("Test Accuracy")
plt.title("KNN: Accuracy vs k")
plt.legend()
plt.show()

```

Best k: 5, Accuracy: 0.9561



8.2 Train Final KNN Model

```

In [34]: knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(X_train_scaled, y_train)

y_pred_knn = knn.predict(X_test_scaled)
print(f"KNN trained with k = {best_k}.")

```

KNN trained with k = 5.

9. Model Evaluation

Metrics: Accuracy, Precision, Recall, F1 Score, Confusion Matrix. Positive class = Malignant (1).

```
In [35]: def get_metrics(y_true, y_pred):
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision": precision_score(y_true, y_pred),
        "Recall": recall_score(y_true, y_pred),
        "F1 Score": f1_score(y_true, y_pred),
    }

    log_metrics = get_metrics(y_test, y_pred_log)
    knn_metrics = get_metrics(y_test, y_pred_knn)

    print("Logistic Regression:", log_metrics)
    print("KNN:", knn_metrics)
```

```
Logistic Regression: {'Accuracy': 0.9649122807017544, 'Precision': 0.975, 'Re
0.9285714285714286, 'F1 Score': 0.9512195121951219}
KNN: {'Accuracy': 0.956140350877193, 'Precision': 0.9743589743589743, 'Recall
0.9047619047619048, 'F1 Score': 0.9382716049382716}
```

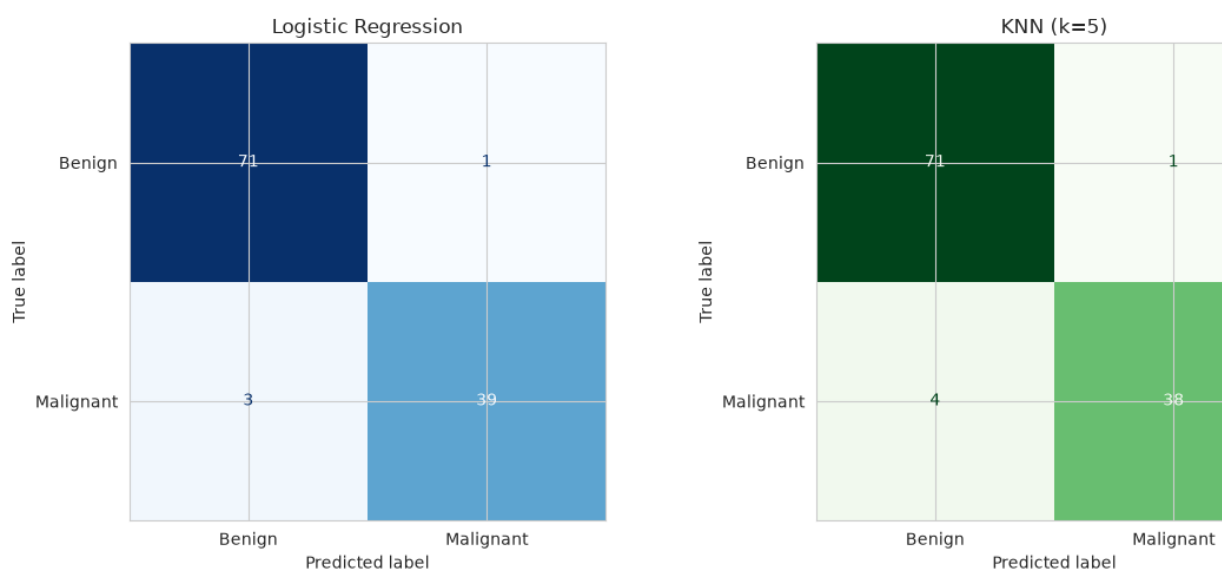
9.1 Confusion Matrices

```
In [36]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    cm_log = confusion_matrix(y_test, y_pred_log)
    ConfusionMatrixDisplay(cm_log, display_labels=["Benign", "Malignant"]).pl
    axes[0].set_title("Logistic Regression")

    cm_knn = confusion_matrix(y_test, y_pred_knn)
    ConfusionMatrixDisplay(cm_knn, display_labels=["Benign", "Malignant"]).pl
    axes[1].set_title(f"KNN (k={best_k})")

    plt.tight_layout()
    plt.show()
```



9.2 Classification Reports

```
In [37]: print("Logistic Regression:\n")
    print(classification_report(y_test, y_pred_log, target_names=["Benign", "Malignant"]))
```



```
print("KNN:\n")
print(classification_report(y_test, y_pred_knn, target_names=["Benign", "
```

Logistic Regression:

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

KNN:

	precision	recall	f1-score	support
Benign	0.95	0.99	0.97	72
Malignant	0.97	0.90	0.94	42
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

10. Comparison Table

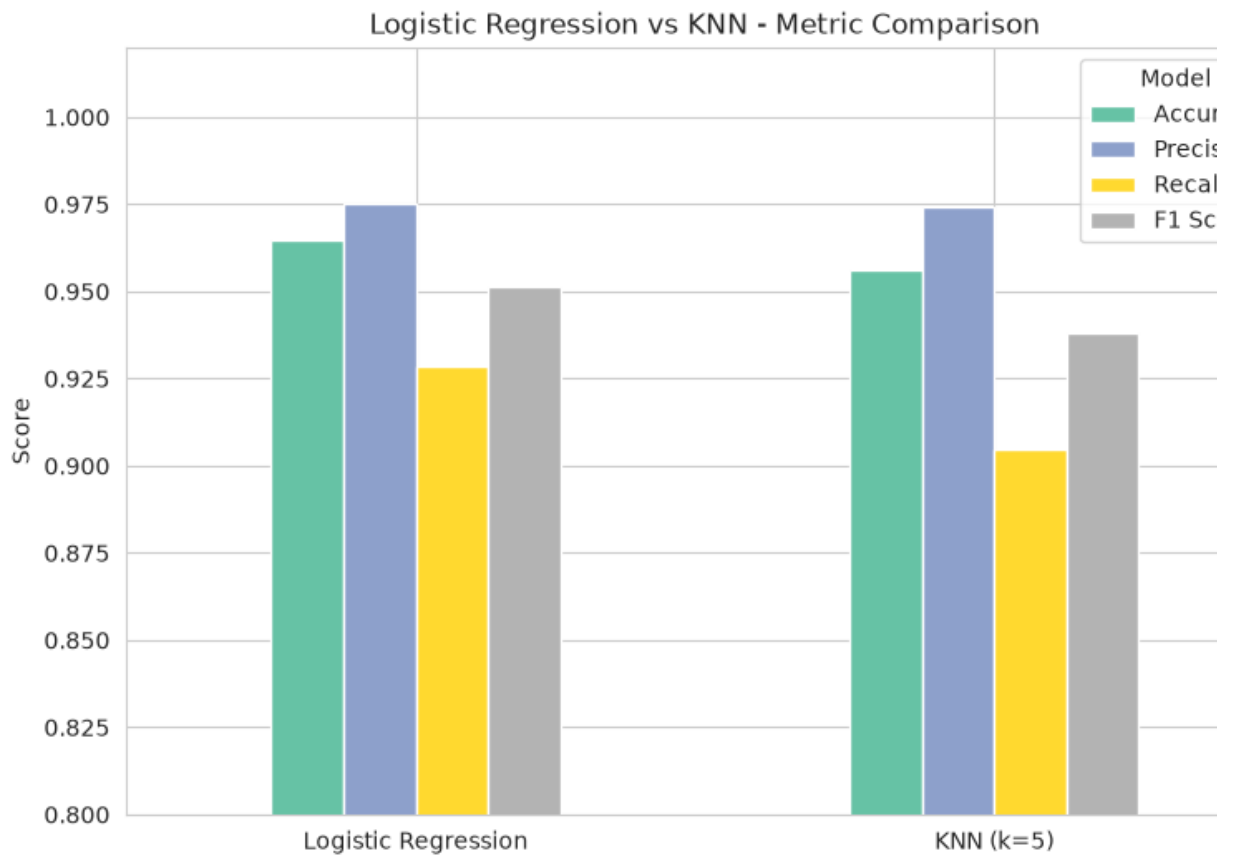
```
In [38]: comparison_df = pd.DataFrame({
    "Logistic Regression": log_metrics,
    f"KNN (k={best_k})": knn_metrics,
}).T.round(4)

comparison_df
```

```
Out[38]:
```

	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.9649	0.9750	0.9286	0.9512
KNN (k=5)	0.9561	0.9744	0.9048	0.9383

```
In [39]: comparison_df.plot(kind="bar", figsize=(9, 6), rot=0, colormap="Set2")
plt.title("Logistic Regression vs KNN - Metric Comparison")
plt.ylabel("Score")
plt.ylim(0.8, 1.02)
plt.legend(title="Model")
plt.show()
```



11. Interpretation of Results

- Logistic Regression accuracy: **96.49%**
- KNN (k=5) accuracy: **95.61%**
- Logistic Regression has higher Precision, Recall and F1 Score too.
- Both models mostly confuse a few Malignant cases as Benign (false negatives) - the more c error type for cancer diagnosis.
- Logistic Regression makes fewer such errors than KNN here.
- Best k for KNN was found to be **5** using the accuracy-vs-k sweep.
- Dataset is linearly well separated, which favors Logistic Regression.
- **Conclusion:** Logistic Regression is the better classifier for this dataset.